

Baysian model

2026-04-17

```
# Bayesian-style PMF in R
```

```
set.seed(42)
```

```
# 1. Load data
```

```
train_path <- "train_ratings.csv"
```

```
val_path   <- "val_ratings.csv"
```

```
test_path  <- "test_ratings.csv"
```

```
train_df <- read.csv(train_path)
```

```
val_df   <- read.csv(val_path)
```

```
test_df  <- read.csv(test_path)
```

```
cat("Train shape:", dim(train_df), "\n")
```

```
## Train shape: 800204 4
```

```
cat("Val shape:", dim(val_df), "\n")
```

```
## Val shape: 100002 4
```

```
cat("Test shape:", dim(test_df), "\n")
```

```
## Test shape: 100003 4
```

```
head(train_df)
```

```
##   user_id movie_id rating timestamp
## 1    3650     1791      3 966459999
## 2    2857     2365      3 972507382
## 3    3789     1034      5 966021326
## 4    4682       648      3 965531092
## 5    5268     2791      5 961171110
## 6    3227     1485      1 969645851
```

```
# 2. Basic checks
```

```
total_n <- nrow(train_df) + nrow(val_df) + nrow(test_df)
```

```
cat("Train proportion:", nrow(train_df) / total_n, "\n")
```

```
## Train proportion: 0.8000368
```

```

cat("Val proportion:", nrow(val_df) / total_n, "\n")

## Val proportion: 0.0999811

cat("Test proportion:", nrow(test_df) / total_n, "\n")

## Test proportion: 0.0999821

cat("Train rating min/max:", min(train_df$rating), max(train_df$rating), "\n")

## Train rating min/max: 1 5

cat("Val rating min/max:", min(val_df$rating), max(val_df$rating), "\n")

## Val rating min/max: 1 5

cat("Test rating min/max:", min(test_df$rating), max(test_df$rating), "\n")

## Test rating min/max: 1 5

# 3. Build mappings
all_user_ids <- unique(c(train_df$user_id, val_df$user_id, test_df$user_id))
all_movie_ids <- unique(c(train_df$movie_id, val_df$movie_id, test_df$movie_id))

n_users <- length(all_user_ids)
n_items <- length(all_movie_ids)

cat("Number of users:", n_users, "\n")

## Number of users: 6040

cat("Number of movies:", n_items, "\n")

## Number of movies: 3706

user2idx <- setNames(seq_along(all_user_ids), all_user_ids)
movie2idx <- setNames(seq_along(all_movie_ids), all_movie_ids)

idx2user <- setNames(all_user_ids, seq_along(all_user_ids))
idx2movie <- setNames(all_movie_ids, seq_along(all_movie_ids))

map_df <- function(df) {
  df$u_idx <- as.integer(user2idx[as.character(df$user_id)])
  df$i_idx <- as.integer(movie2idx[as.character(df$movie_id)])
  df
}

train_df <- map_df(train_df)
val_df <- map_df(val_df)
test_df <- map_df(test_df)

head(train_df)

```

```
##   user_id movie_id rating timestamp u_idx i_idx
## 1    3650     1791      3 966459999     1     1
## 2    2857     2365      3 972507382     2     2
## 3    3789     1034      5 966021326     3     3
## 4    4682      648      3 965531092     4     4
## 5    5268     2791      5 961171110     5     5
## 6     3227     1485      1 969645851     6     6
```

4. Helpers

```
clip_rating <- function(x) {
  pmin(5, pmax(1, x))
}
```

```
rmse <- function(actual, pred) {
  sqrt(mean((actual - pred)^2))
}
```

5. Train PMF model

Model:

$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$

```
fit_bayesian_style_pmf <- function(train_df, val_df, test_df,
                                   n_users, n_items,
                                   k = 20, epochs = 10,
                                   lr = 0.01, reg = 0.02, seed = 42) {
```

```
  set.seed(seed)
```

```
  global_mean <- mean(train_df$rating)
```

latent factors

```
P <- matrix(rnorm(n_users * k, mean = 0, sd = 0.1), nrow = n_users, ncol = k)
```

```
Q <- matrix(rnorm(n_items * k, mean = 0, sd = 0.1), nrow = n_items, ncol = k)
```

biases

```
bu <- rep(0, n_users)
```

```
bi <- rep(0, n_items)
```

```
history <- data.frame(
  epoch = integer(),
  train_rmse = numeric(),
  val_rmse = numeric()
)
```

```
for (ep in 1:epochs) {
  order_idx <- sample(1:nrow(train_df))
```

```
  for (idx in order_idx) {
    u <- train_df$u_idx[idx]
    i <- train_df$i_idx[idx]
    r <- train_df$rating[idx]
```

```
    pred <- global_mean + bu[u] + bi[i] + sum(P[u, ] * Q[i, ])
    err <- r - pred
```

```

# update biases
bu[u] <- bu[u] + lr * (err - reg * bu[u])
bi[i] <- bi[i] + lr * (err - reg * bi[i])

# update latent factors
pu_old <- P[u, ]
qi_old <- Q[i, ]

P[u, ] <- P[u, ] + lr * (err * qi_old - reg * pu_old)
Q[i, ] <- Q[i, ] + lr * (err * pu_old - reg * qi_old)
}

# predictions
train_pred <- global_mean +
  bu[train_df$u_idx] +
  bi[train_df$i_idx] +
  rowSums(P[train_df$u_idx, , drop = FALSE] * Q[train_df$i_idx, , drop = FALSE])

val_pred <- global_mean +
  bu[val_df$u_idx] +
  bi[val_df$i_idx] +
  rowSums(P[val_df$u_idx, , drop = FALSE] * Q[val_df$i_idx, , drop = FALSE])

train_pred <- clip_rating(train_pred)
val_pred <- clip_rating(val_pred)

train_rmse_val <- rmse(train_df$rating, train_pred)
val_rmse_val <- rmse(val_df$rating, val_pred)

history <- rbind(
  history,
  data.frame(epoch = ep, train_rmse = train_rmse_val, val_rmse = val_rmse_val)
)

cat(sprintf("Epoch %d | Train RMSE: %.4f | Val RMSE: %.4f\n",
            ep, train_rmse_val, val_rmse_val))
}

# test rmse
test_pred <- global_mean +
  bu[test_df$u_idx] +
  bi[test_df$i_idx] +
  rowSums(P[test_df$u_idx, , drop = FALSE] * Q[test_df$i_idx, , drop = FALSE])

test_pred <- clip_rating(test_pred)
test_rmse_val <- rmse(test_df$rating, test_pred)

# counts for uncertainty
user_counts <- rep(0, n_users)
item_counts <- rep(0, n_items)

for (u in train_df$u_idx) user_counts[u] <- user_counts[u] + 1
for (i in train_df$i_idx) item_counts[i] <- item_counts[i] + 1

```

```

# seen items
user_seen <- split(train_df$i_idx, train_df$u_idx)
user_seen <- lapply(user_seen, unique)

model <- list(
  global_mean = global_mean,
  bu = bu,
  bi = bi,
  P = P,
  Q = Q,
  history = history,
  test_rmse = test_rmse_val,
  user_counts = user_counts,
  item_counts = item_counts,
  user_seen = user_seen,
  k = k,
  epochs = epochs,
  lr = lr,
  reg = reg,
  seed = seed
)

return(model)
}

# 6. Fit default model
model <- fit_bayesian_style_pmf(
  train_df = train_df,
  val_df = val_df,
  test_df = test_df,
  n_users = n_users,
  n_items = n_items,
  k = 20,
  epochs = 10,
  lr = 0.01,
  reg = 0.02,
  seed = 42
)

```

```

## Epoch 1 | Train RMSE: 0.9153 | Val RMSE: 0.9238
## Epoch 2 | Train RMSE: 0.9023 | Val RMSE: 0.9146
## Epoch 3 | Train RMSE: 0.8958 | Val RMSE: 0.9110
## Epoch 4 | Train RMSE: 0.8889 | Val RMSE: 0.9072
## Epoch 5 | Train RMSE: 0.8783 | Val RMSE: 0.9019
## Epoch 6 | Train RMSE: 0.8639 | Val RMSE: 0.8934
## Epoch 7 | Train RMSE: 0.8489 | Val RMSE: 0.8859
## Epoch 8 | Train RMSE: 0.8339 | Val RMSE: 0.8793
## Epoch 9 | Train RMSE: 0.8195 | Val RMSE: 0.8747
## Epoch 10 | Train RMSE: 0.8063 | Val RMSE: 0.8711

```

```
cat("\nFinal validation RMSE:", tail(model$history$val_rmse, 1), "\n")
```

```
##
```

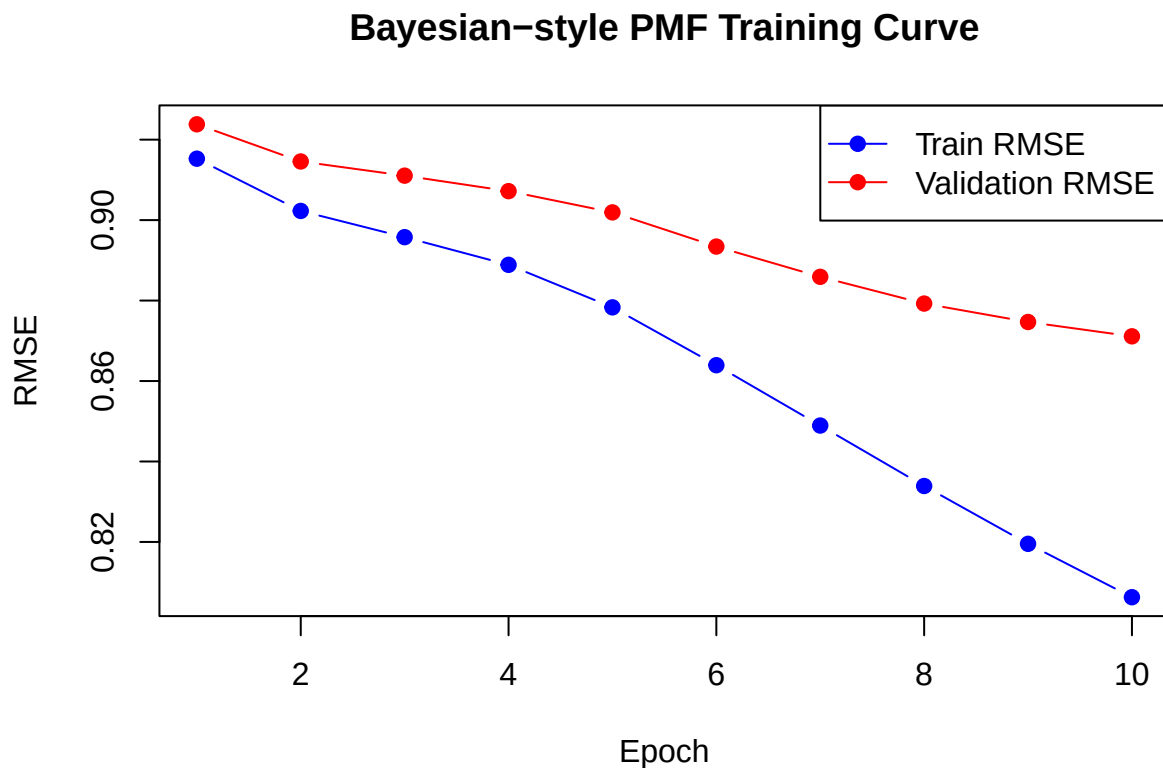
```
## Final validation RMSE: 0.8711135
```

```
cat("Test RMSE:", model$test_rmse, "\n")
```

```
## Test RMSE: 0.8654471
```

```
# 7. Plot training curve
```

```
plot(model$history$epoch, model$history$train_rmse,  
      type = "b", pch = 19, col = "blue",  
      ylim = range(c(model$history$train_rmse, model$history$val_rmse)),  
      xlab = "Epoch", ylab = "RMSE",  
      main = "Bayesian-style PMF Training Curve")  
lines(model$history$epoch, model$history$val_rmse,  
       type = "b", pch = 19, col = "red")  
legend("topright", legend = c("Train RMSE", "Validation RMSE"),  
       col = c("blue", "red"), lty = 1, pch = 19)
```



```
# 8. Single prediction
```

```
predict_mean <- function(model, raw_user_id, raw_movie_id, user2idx, movie2idx) {  
  if (!(as.character(raw_user_id) %in% names(user2idx))) return(NULL)
```

```

if (!(as.character(raw_movie_id) %in% names(movie2idx))) return(NULL)

u <- user2idx[as.character(raw_user_id)]
i <- movie2idx[as.character(raw_movie_id)]

pred <- model$global_mean +
  model$bu[u] +
  model$bi[i] +
  sum(model$P[u, ] * model$Q[i, ])

pred <- clip_rating(pred)
return(pred)
}

# example
sample_user <- train_df$user_id[1]
sample_movie <- train_df$movie_id[1]

pred <- predict_mean(model, sample_user, sample_movie, user2idx, movie2idx)
cat("Sample user_id:", sample_user, "\n")

## Sample user_id: 3650

cat("Sample movie_id:", sample_movie, "\n")

## Sample movie_id: 1791

cat("Predicted rating:", pred, "\n")

## Predicted rating: 2.734201

# 9. Bayesian-style uncertainty
predict_with_uncertainty <- function(model, raw_user_id, raw_movie_id,
                                     user2idx, movie2idx,
                                     n_samples = 300, temp = 0.60) {
  pred <- predict_mean(model, raw_user_id, raw_movie_id, user2idx, movie2idx)
  if (is.null(pred)) return(NULL)

  u <- user2idx[as.character(raw_user_id)]
  i <- movie2idx[as.character(raw_movie_id)]

  user_count <- model$user_counts[u]
  item_count <- model$item_counts[i]

  var_u <- 1 / (1 + user_count)
  var_i <- 1 / (1 + item_count)
  scale <- 0.20 + 0.45 * sqrt(var_u + var_i)

  draws <- rnorm(n_samples, mean = pred, sd = scale)
  draws <- clip_rating(draws)

```

```

stars <- 1:5
weights <- sapply(stars, function(s) {
  exp(-0.5 * ((s - draws) / temp)^2)
})

row_sums <- rowSums(weights)
weights <- weights / row_sums
rating_probs <- colMeans(weights)

list(
  mean_pred = mean(draws),
  low = as.numeric(quantile(draws, 0.025)),
  high = as.numeric(quantile(draws, 0.975)),
  rating_probs = rating_probs,
  draws = draws
)
}

pred_info <- predict_with_uncertainty(
  model, sample_user, sample_movie, user2idx, movie2idx, n_samples = 300
)

cat("Predicted mean rating:", pred_info$mean_pred, "\n")

## Predicted mean rating: 2.74097

cat("95% interval:", pred_info$low, pred_info$high, "\n")

## 95% interval: 2.175662 3.31806

cat("Rating probabilities:\n")

## Rating probabilities:

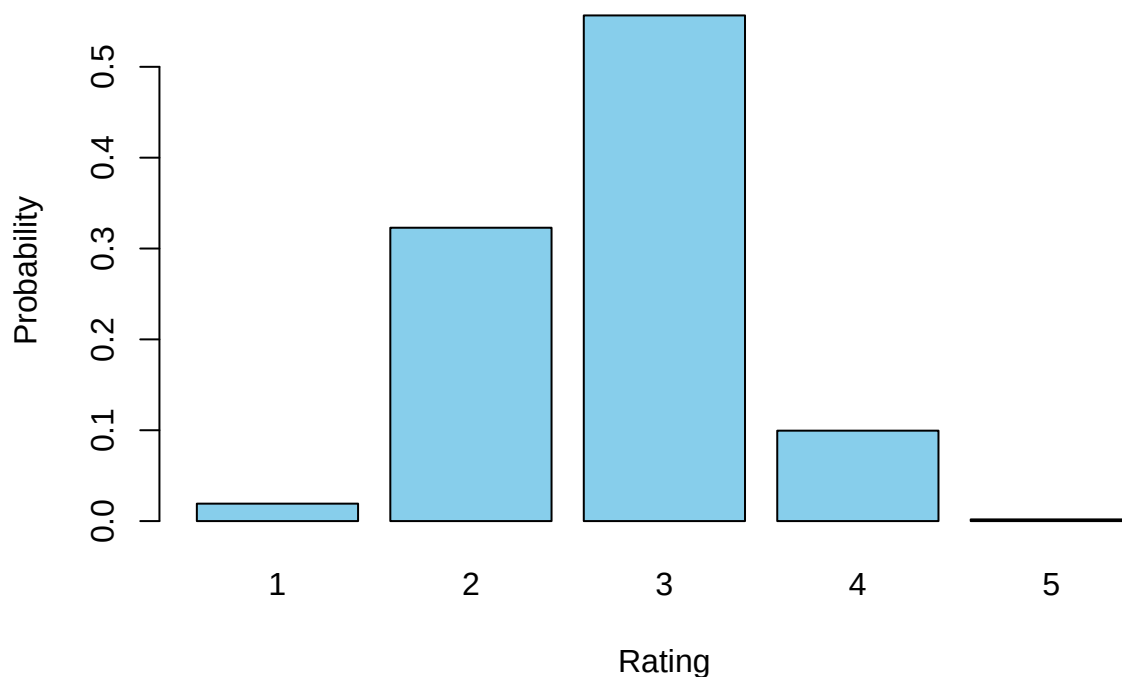
print(data.frame(rating = 1:5, probability = pred_info$rating_probs))

##   rating probability
## 1      1 0.019180582
## 2      2 0.322878819
## 3      3 0.556554885
## 4      4 0.099526215
## 5      5 0.001859498

barplot(pred_info$rating_probs,
  names.arg = 1:5,
  col = "skyblue",
  main = "Probability of Each Discrete Rating",
  xlab = "Rating", ylab = "Probability")

```


Probability of Each Discrete Rating



```
# 10. Top-N recommendation
recommend_top_n <- function(model, raw_user_id, user2idx, idx2movie, top_n = 10) {
  if (!(as.character(raw_user_id) %in% names(user2idx))) return(NULL)

  u <- user2idx[as.character(raw_user_id)]

  seen <- model$user_seen[[as.character(u)]]
  if (is.null(seen)) seen <- integer(0)

  scores <- model$global_mean +
    model$bu[u] +
    model$bi +
    as.vector(model$Q %*% model$P[u, ])

  scores <- clip_rating(scores)

  candidate_idx <- setdiff(1:length(scores), seen)
  candidate_scores <- scores[candidate_idx]

  ord <- order(candidate_scores, decreasing = TRUE)[1:min(top_n, length(candidate_scores))]

  rec_df <- data.frame(
    movie_id = as.numeric(idx2movie[as.character(candidate_idx[ord])]),
    predicted_rating = candidate_scores[ord]
  )
}
```

```

    return(rec_df)
}

rec_df <- recommend_top_n(model, sample_user, user2idx, idx2movie, top_n = 10)
print(rec_df)

```

```

##      movie_id predicted_rating
## 1         1954          5.000000
## 2         2804          4.897823
## 3         1213          4.885636
## 4         2706          4.854719
## 5         2791          4.830276
## 6           50          4.765888
## 7          590          4.699933
## 8         1060          4.693188
## 9         2905          4.690291
## 10          457          4.685287

```

```

# 11. Results summary
results_summary <- list(
  model = "Bayesian-style PMF (R version)",
  seed = 42,
  input_features = "user_id + movie_id only",
  task = "explicit movie rating prediction",
  metric = "RMSE",
  prediction_clip = "[1,5]",
  latent_dim = 20,
  learning_rate = 0.01,
  regularization = 0.02,
  epochs = 10,
  final_val_rmse = as.numeric(tail(model$history$val_rmse, 1)),
  test_rmse = as.numeric(model$test_rmse)
)

print(results_summary)

```

```

## $model
## [1] "Bayesian-style PMF (R version)"
##
## $seed
## [1] 42
##
## $input_features
## [1] "user_id + movie_id only"
##
## $task
## [1] "explicit movie rating prediction"
##
## $metric
## [1] "RMSE"
##
## $prediction_clip

```

```
## [1] "[1,5]"
##
## $latent_dim
## [1] 20
##
## $learning_rate
## [1] 0.01
##
## $regularization
## [1] 0.02
##
## $epochs
## [1] 10
##
## $final_val_rmse
## [1] 0.8711135
##
## $test_rmse
## [1] 0.8654471
```